
MODULE *pipeline_ooo*

EXTENDS *Sequences, instructions_ooo, Integers, TLC, FiniteSets*
 CONSTANTS *locFU*
 VARIABLES *currCycle, prog, _IF, _ID, _RS, _FU, _COM, iMiss, FU, FULat, iMissTmp, FUTmp, FUTmpLat*
 VARIABLES *currCycle2, prog2, _IF2, _ID2, _RS2, _FU2, _COM2, iMiss2, FU2, FULat2, iMissTmp2, FUTmp2*
 VARIABLES *depProg, depProgTmp, commonPre, locWorst*

$vars \triangleq \langle currCycle, prog, _IF, _ID, _RS, _FU, _COM, iMiss, FU, FULat, iMissTmp, FUTmp, FUTmpLat \rangle$
 $vars2 \triangleq \langle currCycle2, prog2, _IF2, _ID2, _RS2, _FU2, _COM2, iMiss2, FU2, FULat2, iMissTmp2, FUTmp2 \rangle$
 $IFs \triangleq \langle _IF, _IF2 \rangle$
 $FUs \triangleq \langle _FU, _FU2 \rangle$
 $progs \triangleq \langle prog, prog2 \rangle$

ASSUME $varIF \in \text{BOOLEAN}$
 ASSUME $locFU \subseteq 1 \dots N_FU$

RECURSIVE *SUM*($-, -$)
 $SUM(seq, n) \triangleq \text{IF } n = 0 \text{ THEN } 0 \text{ ELSE } seq[n] + SUM(seq, n - 1)$

records to deal with potentially equal values that must be distinguishable within the set ($\equiv \text{multiset}$)

RECURSIVE *SetReduce*($-, -, -$)
 $SetReduce(Op(-, -), S, value) \triangleq \text{IF } S = \{\} \text{ THEN } value.val \text{ ELSE LET } s \triangleq \text{CHOOSE } s \in S : \text{TRUEIN } SetReduce(Op, S \setminus \{s\}, [val \mapsto Op(s, value)])$

$Sum(S) \triangleq \text{LET } op(a, b) \triangleq a.val + b.val \text{ IN } SetReduce(op, S, [val \mapsto 0])$

$DomI(i) \triangleq \text{IF } varIF \text{ THEN } Program[i].imiss \text{ ELSE } \{\text{FALSE}\}$
 $DomFU(i) \triangleq prog.rest[i].type$
 $DomFULat(i) \triangleq prog.rest[i].lat$
 $DomDep(i) \triangleq \{\{\}\} \cup \{\{j\} : j \in 1 \dots 3\} \text{ SUBSET } (1 \dots i - 1)$

RECURSIVE *CartProd*($-, -$)
 RECURSIVE *Flatten*($-, -$)
 $CartProd(dom(-), i) \triangleq \text{IF } i = 1 \text{ THEN } dom(1) \text{ ELSE } CartProd(dom, i - 1) \times dom(i)$
 $Flatten(seq, i) \triangleq \text{IF } i = 1 \text{ THEN } \langle seq[1], seq[2] \rangle \text{ ELSE } Append(Flatten(seq[1], i - 1), seq[2])$

1st instance of the pipeline for one execution trace
 $Exec1 \triangleq \text{INSTANCE } pipeline_exec_ooo$

2nd instance of the pipeline with the same input instructions for another execution trace
 $Exec2 \triangleq \text{INSTANCE } pipeline_exec_ooo$
 WITH $currCycle \leftarrow currCycle2,$
 $prog \leftarrow prog2,$
 $_IF \leftarrow _IF2,$

```

    _ID ← _ID2,
    _RS ← _RS2,
    _FU ← _FU2,
    _COM ← _COM2,
    iMiss ← iMiss2,
    FU ← FU2,
    FULat ← FULat2

Init ≜ ∧ IF modeLen ≠ -1 THEN ∧ depProgTmp ∈ CartProd(DomDep, ProgLen)
      ∧ depProg = Flatten(depProgTmp, ProgLen - 1)
    ELSE depProgTmp = ⟨⟩ ∧ depProg = ⟨⟩
  ∧ LET p ≜ IF modeLen ≠ -1 THEN [i ∈ 1 .. ProgLen ↦ [pc ↦ i, type ↦ 1 .. N_FU,
    lat ↦ Lat, dep ↦ depProg[i], imiss ↦ IF varIF THEN BOOLEAN ELSE {FALSE}]]
    ELSE Program
  IN prog = [rest ↦ p, exec ↦ ⟨⟩]
  ∧ prog2 = prog
  ∧ Exec1!Init
  ∧ Exec2!Init
  ∧ commonPre = TRUE
  ∧ locWorst = [i ∈ 1 .. 2 ↦ FALSE]
  ∧ iMissTmp ∈ CartProd(DomI, ProgLen)
  ∧ iMiss = Flatten(iMissTmp, ProgLen - 1)
  ∧ FUTmp ∈ CartProd(DomFU, ProgLen)
  ∧ FU = Flatten(FUTmp, ProgLen - 1)
  ∧ FUTmpLat ∈ CartProd(DomFULat, ProgLen)
  ∧ FULat = Flatten(FUTmpLat, ProgLen - 1)
  ∧ iMissTmp2 ∈ CartProd(DomI, ProgLen)
  ∧ iMiss2 = Flatten(iMissTmp2, ProgLen - 1)
  ∧ FUTmp2 ∈ CartProd(DomFU, ProgLen)
  ∧ FU2 = Flatten(FUTmp2, ProgLen - 1)
  ∧ FUTmpLat2 ∈ CartProd(DomFULat, ProgLen)
  ∧ FULat2 = Flatten(FUTmpLat2, ProgLen - 1)

Next ≜ ∧ Exec1!Next
  ∧ Exec2!Next
  Update of the additional state variables for TALoc
  ∧ commonPre' =
    ∧ commonPre
    Superscalar IFs are actually interchangeable
  ∧ ∀ ex ∈ 1 .. 2 : ∀ k ∈ 1 .. superscal : ∃ kk ∈ 1 .. superscal :
    ∧ IFs[ex][k].PC = IFs[3 - ex][kk].PC
    Do not split locality constraints (discriminate paths at the soonest)
  ∧ IFs[ex][k]'.PC = IFs[ex][k].PC ⇒ IFs[3 - ex][kk]'.PC = IFs[3 - ex][kk].PC
    Similar treatment for FUs since they are gathered in an EX stage...
  ∧ ∀ ex ∈ 1 .. 2 : ∀ k ∈ 1 .. N_FU : ∃ kk ∈ 1 .. N_FU :

```

$$\begin{aligned}
& \wedge FUs[ex][k].PC = FUs[3 - ex][kk].PC \\
& \wedge FUs[ex][k]'.PC = FUs[ex][k].PC \Rightarrow FUs[3 - ex][kk]'.PC = FUs[3 - ex][kk].PC \\
\wedge locWorst' = [i \in 1 \dots 2 \mapsto & \\
\quad \vee & \text{Paths not still comparable and this one already stated as local worst-case} \Rightarrow \text{remains local worst-case} \\
& \wedge \neg commonPre \\
& \wedge locWorst[i] \\
\quad \vee & \text{Still comparable (common prefix)} \\
& \wedge commonPre \\
& \wedge \forall k \in 1 \dots superscal : \forall kk \in 1 \dots superscal : \\
& \quad IFs[i][k].PC = IFs[3 - i][kk].PC \Rightarrow IFs[i][k]'.currLat \geq IFs[3 - i][kk]'.currLat \\
& \wedge \forall k \in 1 \dots N_FU : \forall kk \in 1 \dots N_FU : \\
& \quad FUs[i][k].PC = FUs[3 - i][kk].PC \Rightarrow FUs[i][k]'.currLat \geq FUs[3 - i][kk]'.currLat \\
&] \\
& \wedge \text{UNCHANGED } \langle iMissTmp, FUTmp, FUTmpLat, iMissTmp2, FUTmp2, FUTmpLat2, depProg, depProgTmp, commonPre, locWorst \rangle \\
Spec \triangleq & Init \wedge \Box[Next]_{\langle vars, vars2, depProg, depProgTmp, commonPre, locWorst \rangle} \\
\hline
NotCompleted \triangleq & \exists i \in 1 \dots 2 : Len(progs[i].rest) > 0 \vee Cardinality(Exec1!Done) < ProgLen \vee Cardinality(Exec1!Done) > ProgLen \\
\hline
ProgDone(n) \triangleq & \forall l \in 1 \dots 2 : \exists i \in 1 \dots Len(progs[l].exec) : \\
& progs[l].exec[i].PC.pc = ProgLen \wedge progs[l].exec[i].done \\
ComTime(ex, n) \triangleq & progs[ex].exec[n].comTime \\
\text{Heights of steps} \\
StepHeight(ex, k) \triangleq & \text{IF } k = 1 \text{ THEN } ComTime(ex, 1) \text{ ELSE } ComTime(ex, k) - ComTime(ex, k - 1) \\
NoTASSteps \triangleq & \forall k \in 1 \dots ProgLen - 1 : \forall n \in k + 1 \dots ProgLen : \\
& \text{IF } \wedge ProgDone(n) \\
& \quad \wedge StepHeight(1, k) < StepHeight(2, k) \\
& \quad \Rightarrow ComTime(1, n) \leq ComTime(2, n) \quad \text{As if strict inequality in def} \\
& \text{THEN TRUE} \\
& \text{ELSE } PrintT(\langle \text{"TASSteps encountered (<<First (loc) instr., Second (glob) instr., Delta_alpha, D} \\
& \quad k, n, StepHeight(1, k), StepHeight(2, k), ComTime(2, n) - ComTime(1, n)) \rangle) \wedge \text{FALSE} \\
\text{Intersection in plots} \\
NoTAInter \triangleq & \forall k \in 1 \dots ProgLen - 1 : \forall n \in k + 1 \dots ProgLen : \\
& \text{IF } \wedge ProgDone(n) \\
& \quad \wedge ComTime(1, k) < ComTime(2, k) \\
& \quad \Rightarrow ComTime(1, n) \leq ComTime(2, n) \\
& \text{THEN TRUE} \\
& \text{ELSE } PrintT(\langle \text{"TAInter encountered (<<First (loc) instr., Second (glob) instr., Local diff, Global e} \\
& \quad k, n, ComTime(2, k) - ComTime(1, k), ComTime(2, n) - ComTime(1, n)) \rangle) \wedge \text{FALSE} \\
NoTAInterPart(m) \triangleq & \forall k \in 1 \dots m - 1 : \forall n \in k + 1 \dots m : \\
& \quad \wedge ProgDone(n) \\
& \quad \wedge ComTime(1, k) < ComTime(2, k)
\end{aligned}$$

$$\Rightarrow ComTime(1, n) \leq ComTime(2, n)$$

Locality (pipeline stages)

Prefixes must be the same for comparisons

```

NoTALoc  $\triangleq$  LET  $n \triangleq ProgLen$  IN
  IF  $\wedge ProgDone(n)$ 
     $\wedge \neg locWorst[1]$ 
     $\Rightarrow locWorst[2] \wedge ComTime(2, n) \geq ComTime(1, n)$ 
  THEN TRUE
  ELSE  $PrintT(\langle \text{"TALoc encountered (beta local worst case) } \ll \text{Alpha global time, Beta global time, } ComTime(1, n), ComTime(2, n) \rangle) \wedge FALSE$ 
NoTALocPart( $m$ )  $\triangleq$   $\wedge ProgDone(m)$ 
   $\wedge \neg locWorst[1]$ 
   $\Rightarrow locWorst[2] \wedge ComTime(2, m) \geq ComTime(1, m)$ 

```

Component occupation (parallel inversion)

```

NoTAComp  $\triangleq$  LET  $n \triangleq ProgLen$  IN
  LET  $FUusage(ex, fu) \triangleq FUs[ex][fu].usage$  IN
  LET  $usage(ex) \triangleq Sum(\{[fu \mapsto fu, val \mapsto FUusage(ex, fu)] : fu \in locFU\})$  IN
  IF  $\wedge ProgDone(n)$ 
     $\wedge usage(1) < usage(2)$ 
     $\Rightarrow ComTime(1, n) \leq ComTime(2, n)$ 
  THEN TRUE
  ELSE  $PrintT(\langle \text{"TAComp encountered (} \ll \text{alpha occupation, beta occ, alpha global time, beta } usage(1), usage(2), ComTime(1, n), ComTime(2, n) \rangle) \wedge FALSE$ 

```